

Código auxiliar do multiboot_deploy.py

Funções de validação, persistência e geração de configuração.

Validação do layout:

```
_BOOT_KEY = "boot"
_OSES_KEY = "oses"
_DEFAULT_KEY = "default"
_TIMEOUT_KEY = "timeout"
_OSYSTEM_KEY = "osystem"
_DISTRO_SERIES_KEY = "distro_series"
_HWE_KERNEL_KEY = "hwe_kernel"
_ARCHITECTURE_KEY = "architecture"
_PARTITIONS_KEY = "partitions"
_SIZE_KEY = "size"
_FSTYPE_KEY = "fstype"

_MIN_OS_COUNT = 2
_ALLOWED_FSTYPES = {"ext4", "ext3", "ext2", "xfs", "btrfs",
                    "fat32", "fat16", "ntfs", "swap"}
```

Leitura do arquivo YAML:

```
def read_layout_from_request(request) -> dict:
    # 1. Try request.data
    layout_file = request.data.get("layout")
    if layout_file is not None:
        try:
            raw = layout_file.read() if hasattr(layout_file, 'read') else layout_file
            return yaml.safe_load(raw)
        except yaml.YAMLError as exc:
            raise MAASAPIBadRequest("Invalid YAML: %s" % exc)

    # 2. Try request.POST
    raw = request.POST.get("layout")
    if raw is not None:
        try:
            return yaml.safe_load(raw)
        except yaml.YAMLError as exc:
            raise MAASAPIBadRequest("Invalid YAML: %s" % exc)

    # 3. Try request.FILES (multipart from CLI @= syntax)
    layout_file = request.FILES.get("layout")
    if layout_file is not None:
        try:
            return yaml.safe_load(layout_file.read())
        except yaml.YAMLError as exc:
            raise MAASAPIBadRequest("Invalid YAML: %s" % exc)

    raise MAASAPIBadRequest("No layout file provided.")
```

Validação de estrutura:

```
def validate_layout(layout_data: dict) -> None:
    if not isinstance(layout_data, dict):
        raise MAASAPIValidationError("Layout must be a dict.")
    if _BOOT_KEY not in layout_data:
        raise MAASAPIValidationError("Layout must contain '%s'." % _BOOT_KEY)
```

```

if _OSES_KEY not in layout_data:
    raise MAASAPIValidationError("Layout must contain '%s'." % _OSES_KEY)

boot = layout_data[_BOOT_KEY]
oses = layout_data[_OSES_KEY]

if not isinstance(oses, list):
    raise MAASAPIValidationError("'%s' must be a list." % _OSES_KEY)
if len(oses) < _MIN_OS_COUNT:
    raise MAASAPIValidationError("At least %d OS entries required." %
_Min_OS_Count)
if not isinstance(boot, dict):
    raise MAASAPIValidationError("'%s' must be a dict." % _BOOT_KEY)

default_index = boot.get(_DEFAULT_KEY, 0)
if not isinstance(default_index, int) or default_index < 0:
    raise MAASAPIValidationError("'%.s' must be >= 0." % (_BOOT_KEY,
_Default_Key))
if default_index >= len(oses):
    raise MAASAPIValidationError("'%.s' out of range." % (_BOOT_KEY,
_Default_Key))

timeout = boot.get(_TIMEOUT_KEY, 5)
if not isinstance(timeout, int) or timeout < 0:
    raise MAASAPIValidationError("'%.s' must be >= 0." % (_BOOT_KEY,
_Timeout_Key))

for i, os_entry in enumerate(oses):
    validate_os_entry(os_entry, i)

def validate_os_entry(os_entry: dict, index: int) -> None:
    prefix = "'oses[%d]'" % index
    if not isinstance(os_entry, dict):
        raise MAASAPIValidationError("%s must be a dict." % prefix)
    if _OSYSTEM_KEY not in os_entry:
        raise MAASAPIValidationError("%s missing '%s'." % (prefix, _OSYSTEM_KEY))
    if _DISTRO_SERIES_KEY not in os_entry:
        raise MAASAPIValidationError("%s missing '%s'." % (prefix,
_Distro_Series_Key))
    if _PARTITIONS_KEY not in os_entry:
        raise MAASAPIValidationError("%s missing '%s'." % (prefix, _PARTITIONS_KEY))
    partitions = os_entry[_PARTITIONS_KEY]
    if not isinstance(partitions, dict):
        raise MAASAPIValidationError("%s '%s' must be a dict." % (prefix,
_Partitions_Key))
    if "/" not in partitions:
        raise MAASAPIValidationError("%s '%s' must contain '/'." % (prefix,
_Partitions_Key))
    for mount_point, part_config in partitions.items():
        validate_partition(part_config, mount_point, prefix, os_entry[_OSYSTEM_KEY])

def validate_partition(part_config, mount_point, os_prefix, osystem):
    prefix = "%s '%s'['%s']" % (os_prefix, _PARTITIONS_KEY, mount_point)
    if not isinstance(part_config, dict):

```

```

        raise MAASAPIValidationError("%s must be a dict." % prefix)
    if _SIZE_KEY not in part_config:
        raise MAASAPIValidationError("%s missing '%s'." % (prefix, _SIZE_KEY))
    if _FSTYPE_KEY not in part_config:
        raise MAASAPIValidationError("%s missing '%s'." % (prefix, _FSTYPE_KEY))
    fstype = part_config[_FSTYPE_KEY]
    if fstype not in _ALLOWED_FSTYPES:
        raise MAASAPIValidationError("%s invalid fstype '%s'." % (prefix, fstype))
    if fstype == "ntfs" and osystem != "windows":
        raise MAASAPIValidationError("ntfs only for windows.")

```

Composição de preseed individual:

```

def compose_os_config(request, machine, os_entry: dict, index: int) -> str:
    osystem = os_entry[_OSYSTEM_KEY]
    series = os_entry[_DISTRO_SERIES_KEY]
    old_osystem = machine.osystem
    old_series = machine.distro_series
    try:
        machine.osystem = osystem
        machine.distro_series = series
        merged = get_curtin_merged_config(request, machine)
    finally:
        machine.osystem = old_osystem
        machine.distro_series = old_series
    merged.pop("storage", None)
    return yaml.safe_dump(merged)

```

Configuração de disco unificada:

```

def build_storage_config(layout: dict, machine=None) -> dict:
    config = []
    oses = layout.get(_OSES_KEY, [])

    # Detecta disco real do hardware comissionado
    disk_name = "sda"
    if machine:
        if machine.boot_disk:
            disk_name = machine.boot_disk.name or "sda"
        else:
            pbd = PhysicalBlockDevice.objects.filter(
                node_config__node=machine
            ).first()
            if pbd:
                disk_name = pbd.name or "sda"

    config.append({"id": disk_name, "type": "disk", "ptable": "gpt",
                  "path": "/dev/%s" % disk_name, "name": disk_name})

    # ESP compartilhada
    config.append({"id": "esp", "type": "partition",
                  "device": disk_name, "size": "512M", "flag": "boot"})
    config.append({"id": "esp_fs", "type": "format",
                  "volume": "esp", "fstype": "fat32"})
    config.append({"id": "esp_mnt", "type": "mount",
                  "device": "esp_fs", "path": "/boot/efi"})

    # Partições por OS

```

```

part_num = 2
first_linux = True
for i, entry in enumerate(oses):
    is_windows = entry.get(_OSYSTEM_KEY, "") == "windows"
    partitions = entry.get(_PARTITIONS_KEY, {})
    for mount_point, part_spec in partitions.items():
        if mount_point == "/boot/efi":
            continue
        part_id = f"os{i}_p{part_num}"
        config.append({"id": part_id, "type": "partition",
                       "device": disk_name, "size": part_spec["size"]})
        fs_id = f"{part_id}_fs"
        config.append({"id": fs_id, "type": "format",
                       "volume": part_id, "fstype": part_spec[_FSTYPE_KEY]})
        if not is_windows and first_linux:
            config.append({"id": f"{part_id}_mnt", "type": "mount",
                           "device": fs_id, "path": mount_point})
        part_num += 1
    if not is_windows and first_linux:
        first_linux = False

return {"version": 1, "config": config}

```

Persistência:

```

def load_layout(machine) -> Optional[dict]:
    deployment, os_entries, _ = \
        service_layer.services.multiboot_deployments.get_layout(machine.id)
    if deployment is None:
        return None
    oses_list = []
    for os_entry, partitions in os_entries:
        parts = {p.mount_point: {"size": p.size, "fstype": p.fstype}
                  for p in partitions}
        oses_list.append({
            "osystem": os_entry.osystem,
            "distro_series": os_entry.distro_series,
            "partitions": parts,
        })
    return {"boot": {"default": deployment.default_os_index,
                     "timeout": deployment.boot_timeout},
            "oses": oses_list}

```

@transaction.atomic

```

def store_layout(node: Node, layout_data: dict) -> None:
    boot = layout_data.get("boot", {})
    oses = layout_data.get("oses", [])
    oses_data = []
    for i, os_entry in enumerate(oses):
        partitions_data = []
        for mp, pc in os_entry.get("partitions", {}).items():
            partitions_data.append({
                "mount_point": mp,
                "size": _parse_size(pc["size"]),
                "fstype": pc.get("fstype", "ext4"),
            })

```

```

        oses_data.append({
            "osystem": os_entry["osystem"],
            "distro_series": os_entry["distro_series"],
            "partitions": partitions_data,
        })
    service_layer.services.multiboot_deployments.store_layout(
        node_id=node.id,
        default_os_index=boot.get("default", 0),
        boot_timeout=boot.get("timeout", 5),
        oses_data=oses_data,
    )

```

```

def delete_layout(machine: Node) -> None:
    service_layer.services.multiboot_deployments.delete_layout(machine.id)
    NodeMetadata.objects.filter(
        node=machine, key__startswith="multi_boot_"
    ).delete()

```

Geração do instalador (exceto orquestrador):

```

def generate_multi_boot_installer(layout, machine, os_configs):
    # 1. Resolve URLs de imagem
    urls = []
    for entry in layout["oses"]:
        old_os = machine.osystem
        old_series = machine.distro_series
        try:
            machine.osystem = entry["osystem"]
            machine.distro_series = entry["distro_series"]
            raw_url = get_curtin_installer_url(machine)
        finally:
            machine.osystem = old_os
            machine.distro_series = old_series
        clean_url = re.sub(r'^[a-z][a-z0-9-]*:', '', raw_url) \
            if re.match(r'^[a-z][a-z0-9-]*:http', raw_url) else raw_url
        urls.append(clean_url or "cp:///media/root-ro")

    # 2. Valida espaço em disco
    ESP_BYTES = 512 * 1024 * 1024
    total = ESP_BYTES + sum(
        _parse_size(p["size"])
        for e in layout["oses"]
        for p in e["partitions"].values()
    )
    disk_size = (machine.boot_disk.size or 0) if machine.boot_disk else \
        (PhysicalBlockDevice.objects.filter(
            node_config__node=machine).first().size or 0)
    if disk_size > 0 and total > disk_size:
        raise MAASAPIValidationError(
            "Espaço requerido (%.1f GB) excede o disco (%.1f GB).\"
            % (total / 1e9, disk_size / 1e9)
        )

    # 3. Constrói storage mestre
    master_storage = build_storage_config(layout, machine)

```

```

# 4. Extraí config de rede do primeiro preseed
first_cfg = yaml.safe_load(os_configs[0])
net_config = first_cfg.get("network_commands") if first_cfg else None

# 5. Mescla e limpa
config_yamls = []
for i, yaml_str in enumerate(os_configs):
    cfg = yaml.safe_load(yaml_str) or {}
    if i == 0:
        merge_config(cfg, {"storage": master_storage})
    else:
        cfg["grub_device"] = False
        for key in ("datasource", "cloud_config", "reporting", "rsyslog"):
            cfg.pop(key, None)
    if net_config:
        cfg["network_commands"] = net_config
    config_yamls.append(yaml.safe_dump(cfg))

# 6. Gera orquestrador e empacota
orch = _generate_orchestrator(
    oses=layout["oses"], urls=urls,
    default_index=layout["boot"].get("default", 0),
    timeout=layout["boot"].get("timeout", 5),
    system_id=machine.system_id,
    metadata_url=str(machine.boot_cluster_ip),
)
config_files = [
    (f"configs/config-{i:03d}.cfg", y)
    for i, y in enumerate(config_yamls)
] + [("multi-boot.sh", orch)]

return pack(command=["/bin/bash", "multi-boot.sh"],
            add_files=config_files)

```